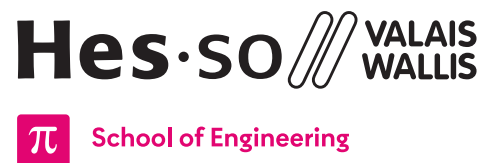




Game of Life (GoL)

Vorlesung Computerarchitektur (CAr)



Orientierung: [Informatik und Kommunikationswissenschaften \(ISC\)](#)

Spezialisierung: Data Engineering (DE)

Kurs: [Computerarchitektur \(CAr\)](#)

Autoren: [Silvan Zahno](#), [Axel Amand](#), [Rémy Borgeat](#)

Datum: 05.08.2026

Version: v1.0



Inhalt

1 Einführung	3
2 Spezifikation	4
2.1 Minimale Spezifikation	6
2.2 Regeln	6
2.2.1 Toroidale Gitter	7
2.2.2 Tipps	7
3 Software	9
3.1 Grids und Zellenrepräsentation	9
3.1.1 Grid	9
3.1.2 Zelle	9
3.2 Aufgaben	10
3.2.1 gol_init	10
3.2.2 gol_step	10
3.3 Kompilieren und flashen	11
3.4 Tests	12
3.4.1 Hardware In The Loop (HITL)	12
3.5 Verwendung des GoL Viewers	13
3.5.1 Einrichtung	13
3.5.2 Anzeige des Grids	14
3.5.3 Bearbeitung des Grids	14
4 Bewertung	16
Anhang	17
1 Debugging (Optional)	17
1.1 Einrichtung	17
1.1.1 Erweiterungen	17
1.1.2 Treiber	17
1.1.3 Debug	18
Glossar	21



1 Einführung

Ziel des Projekts ist es, das am Ende des Semesters erworbene Wissen direkt anhand eines praktischen Beispiels anzuwenden.

Ziel ist es, **Conways Game of Life (GoL)** zu implementieren, ein zelluläres Automaten-System, das auf 4 einfachen Regeln basiert und „von selbst“ agiert - *auch bekannt als Nullspieler-Spiel* -.

Das **Spiel des Lebens** wurde 1970 vom britischen Mathematiker John Conway entwickelt. Ein Raster besteht aus Zellen, die entweder lebendig oder tot sein können. Die Entwicklung des Rasters basiert auf 4 einfachen Regeln, die auf jede Zelle basierend auf ihren Nachbarn angewendet werden - siehe **Abschnitt 2.2**.

Das Spiel verwendet eine Desktop-Anwendung, um ein Raster von Zellen zu erstellen, das dann an ein **ESP-RS devboard** gesendet wird.

Der Mikrocontroller wendet die Regeln des Spiels auf das Raster an und sendet das Ergebnis zurück an die Anwendung, um das System live zu sehen. Die rote **Light Emitting Diode (LED)** auf der Platine wechselt jedes Mal, wenn das Raster aktualisiert wird, um zu zeigen, dass es läuft. Dieses System ist in der Abbildung **Abbildung 1** dargestellt:

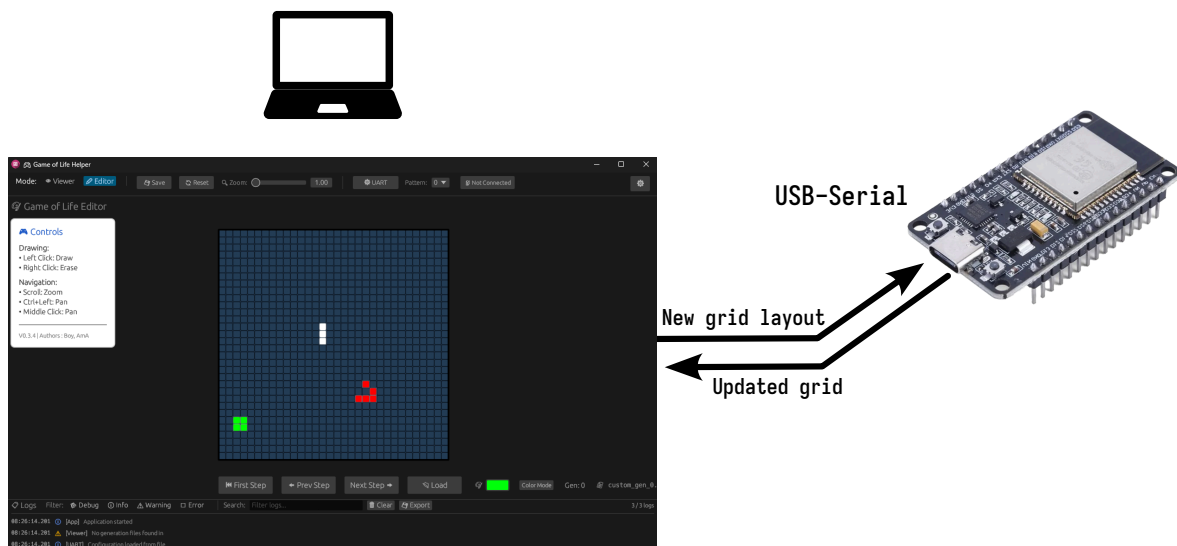


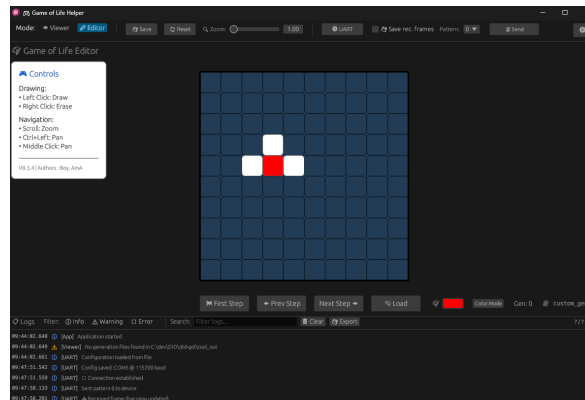
Abbildung 1 - Hardwareaufbau GoL

Das Ziel ist es, die Rastergenerierung in Assembler gemäss der Spezifikation **Abschnitt 2** zu implementieren.



2 Spezifikation

Das System verwendet den **GoL Viewer**, eine Desktop-Anwendung, um Raster von Zellen zu erstellen und anzuzeigen



Die Zellen werden dann an ein **ESP-RS devboard** gesendet.

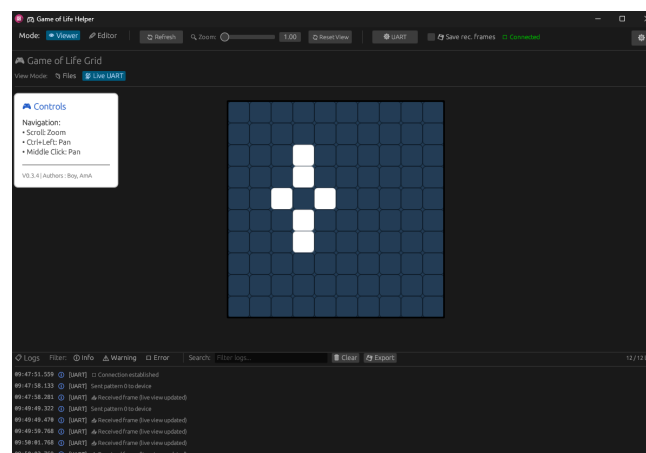
Diese Karte beherbergt einen **ESP32-C3**-Chip, der einen ein Kern 32-Bit **5-stages Reduced Instruction Set Computer architecture, open-sourced (RISC-V)** Mikrocontroller hostet, der die **RV32IMC**-Instruktionsmengen unterstützt.

Es läuft eine auf **Embedded Rust** basierende Anwendung, die Rust und teilweise **RISC-V** Assembly-Code an Schlüsselstellen verwendet, wie im Diagramm **Abbildung 4** gezeigt.

Der Assembly-Code ist in **zwei Teile** unterteilt:

1. **gol_init**: richtet ein Standardrastermuster durch den ESP32-C3
2. **gol_step**: empfängt ein Raster vom **GoL Viewer** und muss die **GOL**-Regeln darauf anwenden, dann an den Viewer senden

Wenn Raster empfangen werden, zeigt die Anwendung **GoL Viewer** sie auf dem Bildschirm an:



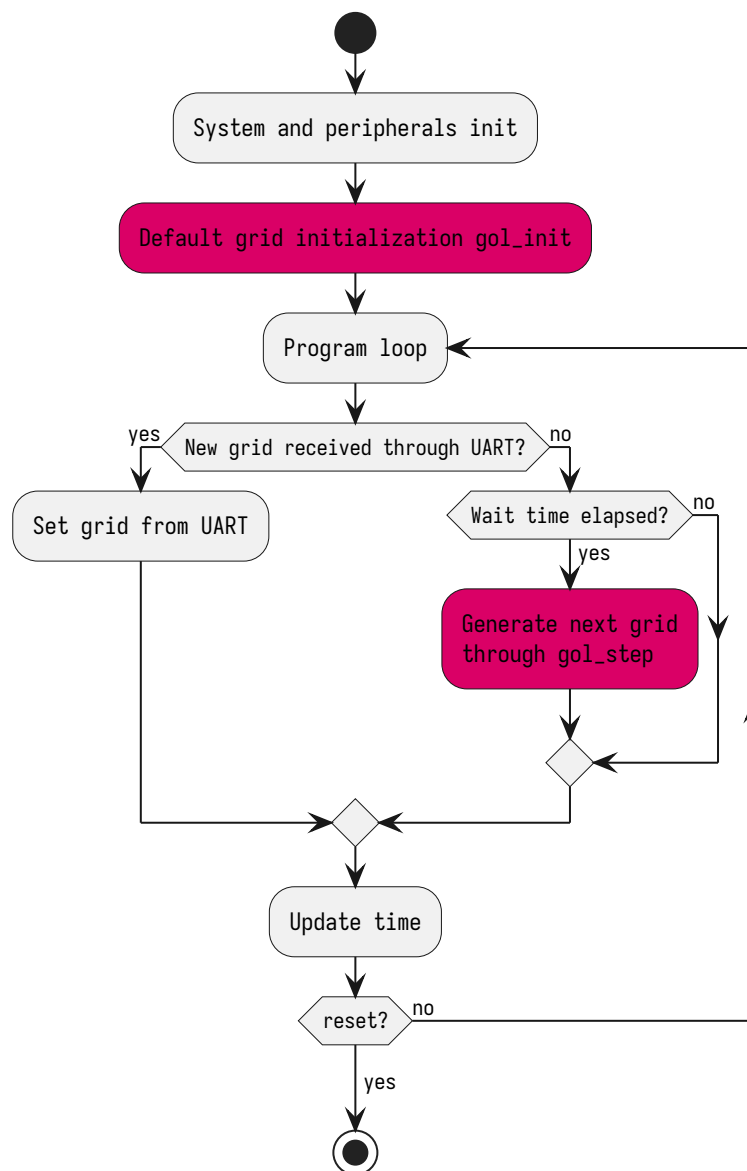


Abbildung 4 - Systemflussdiagramm für die Game of Life-Implementierung



Die **C**-Erweiterung des **RV32IMC** Befehlssatzes definiert komprimierte Anweisungen, die insbesondere auf eingebetteten Systemen mit begrenztem Speicher nützlich sind. In diesem Projekt kann diese **Instruction Set Architecture (ISA)** beim Schreiben von direktem Assembly-Code ignoriert werden.



2.1 Minimale Spezifikation

Das System muss die folgenden Funktionalitäten implementieren:

- Beim Start in **gol_init** muss das System ein Standardrastermuster auf dem Bildschirm anzeigen, ohne dass eine Benutzerinteraktion vom **GoL Viewer** erforderlich ist. Das Raster muss 32x32 Zellen enthalten und ein *Blinker-Muster* im Zentrum enthalten - [Abbildung 5](#).
- Bei jedem Aufruf von **gol_step** muss das Raster gemäss den **GoL**-Regeln [Abschnitt 2.2](#) aktualisiert und an die Anwendung **GoL Viewer** zurückgesendet werden, um es anzuzeigen. Die Regeln müssen auf eine *toroidale* Weise angewendet werden, was bedeutet, dass das Raster an seinen Rändern umschlägt - siehe [Abschnitt 2.2.1](#).
- Der Code muss Konventionen folgen und Funktionsaufrufe verwenden, um Code-Duplikation zu vermeiden und die Lesbarkeit und das Debugging zu verbessern. Der Code muss kommentiert werden, um Funktionen und Logik hinter der Implementierung zu erklären.
- Ein Bericht muss gemäss [Abschnitt 4](#) geschrieben werden.
- Ein grafisches Zusammenfassungsdiagramm des Codeflusses muss ähnlich wie [Abbildung 2](#) bereitgestellt werden, das die Hauptschritte von **gol_init** mit Entscheidungen und Funktionsaufrufen zeigt.
- Ein grafisches Zusammenfassungsdiagramm des Codeflusses muss ähnlich wie [Abbildung 2](#) bereitgestellt werden, das die Hauptschritte von **gol_step** mit Entscheidungen und Funktionsaufrufen zeigt.
- Es ist keine KI-generierte Inhalte im Code oder Bericht erlaubt. Der Code und der Bericht müssen vom Studenten geschrieben werden.

2.2 Regeln

GoL ist ein System von zellulären Automaten, das aus einem Raster von Zellen besteht, von denen jede entweder *lebendig* oder *tot* ist. Die Entwicklung des Rasters folgt vier einfachen Regeln, die auf die aktuelle Zelle und ihre umliegenden Nachbarn basieren:

1. Unterbevölkerung

Jede lebende Zelle mit weniger als zwei (< 2) lebenden Nachbarn *stirbt*



2. Überleben

Jede lebende Zelle mit zwei ($= 2$) lebenden Nachbarn *überlebt* in die nächste Generation



3. Überbevölkerung

Jede lebende Zelle mit mehr als drei (> 3) lebenden Nachbarn *stirbt*



4. Reproduktion

Jede tote Zelle mit genau drei ($= 3$) lebenden Nachbarn wird *lebendig*



Diese einfachen Regeln erzeugen faszinierende emergente Verhaltensweisen, einschliesslich stabiler Muster, Oszillatoren und mobiler Strukturen, die als „Gleiter“ bezeichnet werden, wie in [Abbildung 5](#) gezeigt:

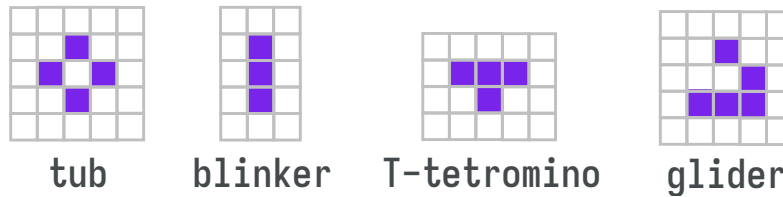


Abbildung 5 - Interessante Muster in GoL : stabile Strukturen, Oszillatoren und Glieder

Das Spiel kann unter <https://playgameoflife.com> oder <https://lifeee.netlify.app> ausprobiert werden.

2.2.1 Toroidale Gitter

Ein **toroidales** Gitter ist ein Gitter, bei dem sich die Kanten umschlagen und eine „unendliche“ Oberfläche ohne Grenzen schaffen.

Auf dieses Projekt angewendet bedeutet dies, dass einige Nachbarn von Zellen an den Extremitäten des Gitters auf der gegenüberliegenden Seite erscheinen.

Als Beispiel nehmen wir das folgende 6x6-Gitter und überprüfen die Nachbarn der Zelle (0,0):

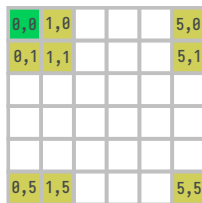


Abbildung 6 - Nachbarn der Zelle (0, 0)

1. Die linken Nachbarn überlaufen nach rechts aus dem Gitter
2. Die oberen Nachbarn überlaufen nach unten aus dem Gitter
3. Der obere linke Nachbar überläuft nach unten rechts aus dem Gitter

Verallgemeinert sind die Nachbarn einer Zelle an der Position (x, y) in einem Gitter der Grösse (X_Size, Y_Size):

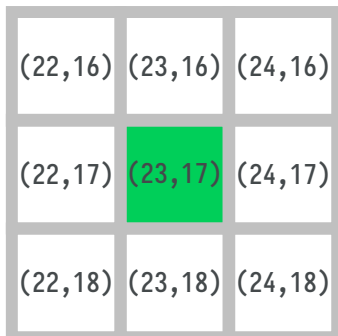


Abbildung 7 - Nachbarn der Zelle (23, 17)

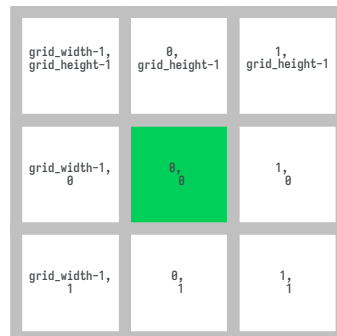


Abbildung 8 - Nachbarn der Zelle (0, 0)

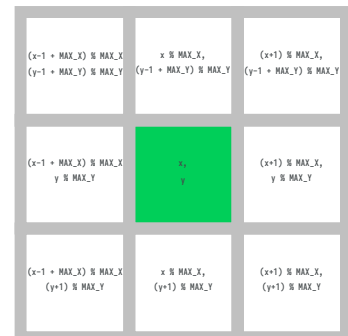


Abbildung 9 - Nachbarn einer generischen Zelle (x, y)

2.2.2 Tipps

1. Beginnen Sie damit, die GoL -Spiele im Web zu testen, um die Regeln und deren Funktionsweise zu verstehen.
2. Beginnen Sie damit, den Codefluss, die Funktionen, Entscheidungen ... zu definieren und zeichnen Sie ein Diagramm ähnlich wie Abbildung 2. Dies wird Ihnen helfen, das System zu verstehen und wie es implementiert werden kann.
3. Implementieren Sie die Funktion `gol_init`, um mit Lese- und Schreibvorgängen im Speicher zu spielen. Ersetzen Sie die Zeile `unsafe { gol_step(cur, nxt, COLS as u32, ROWS as u32) }`;



in `main.rs` durch `rust_send_grid(cur as *const u32, CELLS as u32);`, um dieses Raster im GoL Viewer anzuzeigen und zu visualisieren.

4. Stellen Sie den Rust-Code `unsafe { gol_step...` wieder her und implementieren Sie die Funktion `gol_step`, um die Regeln des Spiels auf das Raster anzuwenden. Gehen Sie in kleinen Schritten vor und testen Sie häufig, indem Sie Ihren Code ausführen und die Entwicklung im GoL Viewer anzeigen. Wenn nötig, iterieren Sie Schritt für Schritt mit dem Debugger - siehe [Abschnitt 1](#).
5. Sobald alles eingerichtet ist, führen Sie die Tests aus, um sicherzustellen, dass sowohl die Generierung als auch die toroidalen Regeln korrekt implementiert sind - siehe [Abschnitt 3.4](#).



3 Software



Der zu implementierende **Assembly (ASM)** -Code befindet sich unter `src/asm/gol.s` und wird vom Rust-Code in `src/bin/main.rs` aufgerufen.

3.1 Grids und Zellenrepräsentation

Da der Assemblercode mit vorgegebenem Rust-Code gemischt ist, ist es wichtig, zunächst einige wichtige Begriffe zu verstehen.

3.1.1 Grid

Die Grids sind in `grid.rs` wie folgt definiert:

```
pub static mut GRID_X: [u32; WIDTH * HEIGHT] = [0u32; WIDTH * HEIGHT];
```

Im Speicher wird ein Grid als zusammenhängendes 1D-Array von **Breite * Höhe** Zellen gespeichert, kann jedoch als 2D-Array betrachtet werden, wobei jede Zelle durch ihre (x, y)-Koordinaten angegeben wird.

Nehmen wir zum Beispiel ein 3x3-Zellen-Grid:

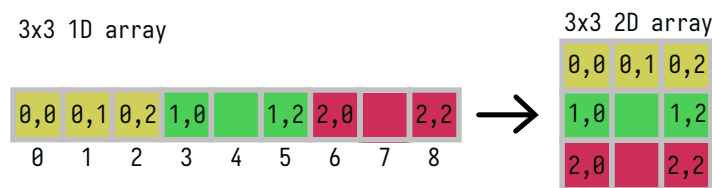


Abbildung 10 - Grid im Speicher



Interne, werden zwei Grids **A** und **B** verwendet, insbesondere für [Abschnitt 3.2.2](#). Dies ermöglicht es, die nächste Generation der Grid in **B** zu berechnen, während die aktuelle Generation in **A** gelesen wird, ohne Daten zu verlieren. Sobald die Berechnung abgeschlossen ist, werden die beiden Grids ausgetauscht.

3.1.2 Zelle

Wie im [vorherigen Codeausschnitt 1](#) zu sehen ist, wird das Grid als Array von 32-Bit-Unsignierten Integern definiert.

Jede Zelle wird in Wirklichkeit als 32Bit **xxRRGGBB**-Wert dargestellt, wobei **xx** ein reserviertes Byte ist (fixiert auf **0x00**) und **RRGGBB** die Farbe der Zelle im hexadezimalen Format ist.

Diese Farbe wird auch vom **GoL Viewer** verwaltet und kann für erweiterte Funktionen wie die Implementierung eines [Rainbow Game of Life](#) verwendet werden, bei dem jede Zelle beim Geborenwerden Farben erbt.

Eine **tote** Zelle wird dadurch bestimmt, dass alle ihre Bits **Red Green Blue (RGB)** auf 0 gesetzt sind, während alle anderen Werte von **RGB** als **lebende** Zellen betrachtet werden:

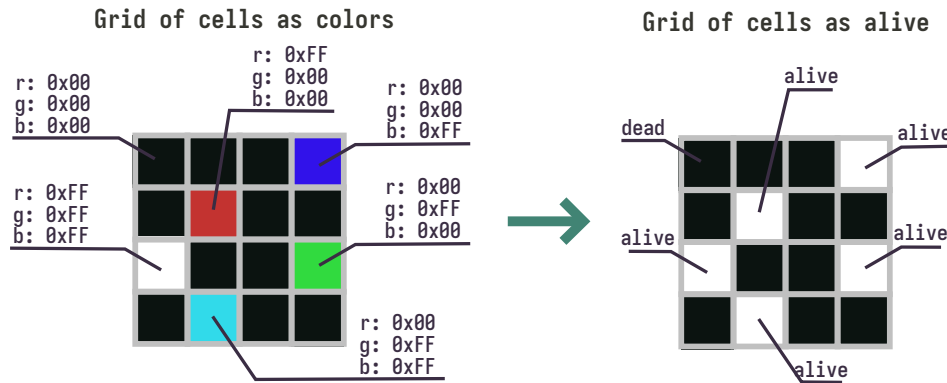


Abbildung 11 - Status einer lebenden oder toten Zelle

3.2 Aufgaben

3.2.1 gol_init

Die Funktion **gol_init** wird beim Start aufgerufen und muss das Grid mit dem Standardmuster initialisieren - einem *Blinker* im Zentrum des Grids - [Abbildung 5](#).

Sie ist definiert als **fn gol_init(data_ptr: *mut u32, cols: u32, rows: u32)**; wobei:

- **data_ptr** die Speicheradresse des Grids, Zelle (0, 0) enthält. Achten Sie darauf, nur auf zulässige Speicherorte zuzugreifen, da keine Grenzprüfung erfolgt.
- **cols** ist die Gesamtzahl der Spalten im Grid
- **rows** ist die Gesamtzahl der Zeilen im Grid



1. Zeichnen Sie ein Flussdiagramm, das die Schritte zur Initialisierung des Grids mit einem Blinker-Muster in der Mitte darstellt.
2. Schreiben Sie Assemblercode, um das gesamte Grid zu durchlaufen, um zu entscheiden, ob eine Zelle als **tot** (0xXX000000) oder **lebendig** (jeder andere Wert, abhängig davon, ob Farben verwendet werden oder nicht) gesetzt werden soll.
3. Initialisieren Sie das Standard-Grid-Muster beim Start, indem Sie einen Blinker - [Abbildung 5](#) - in der Mitte platzieren.

3.2.2 gol_step

Die Funktion **gol_step** wird jedes Mal aufgerufen, wenn die nächste Generation des Grids berechnet werden muss.

Sie ist definiert als **fn gol_step(in_data_ptr: *mut u32, out_data_ptr: *mut u32, cols: u32, rows: u32)**; wobei:

- **in_data_ptr** die Speicheradresse des Eingabegrids, Zelle (0, 0) enthält. Achten Sie darauf, nur auf zulässige Speicherorte zuzugreifen, da keine Grenzprüfung erfolgt. Lesen Sie nur davon.
- **out_data_ptr** die Speicheradresse des Ausgabegrids, Zelle (0, 0) enthält. Achten Sie darauf, nur auf zulässige Speicherorte zuzugreifen, da keine Grenzprüfung erfolgt. Schreiben Sie nur darauf.
- **cols** ist die Gesamtzahl der Spalten im Grid
- **rows** ist die Gesamtzahl der Zeilen im Grid

Das Rezept zur Berechnung der nächsten Generation lautet wie folgt:



1. Iterieren Sie über alle Eingabezellen. Für jede:
 - Iterieren Sie über alle Nachbarn der aktuellen Zelle und zählen Sie, wie viele **lebendig** sind
 - Wenden Sie die Regeln von [Abschnitt 2.2](#) an, um zu bestimmen, ob die aktuelle Zelle in der nächsten Generation lebendig oder tot ist
 - Registrieren Sie den neuen Wert im Ausgabegrid an denselben Koordinaten wie die aktuelle Zelle
2. Sobald das Grid vollständig berechnet ist, senden Sie es über **Universal Asynchronous Receiver Transmitter (UART)**, indem Sie die Hilfsfunktion `fn rust_send_grid(ptr: *const u32, count: u32)`; aufrufen, wobei:
 - **ptr** die Speicheradresse des Ausgabegrids, Zelle **(0, 0)** enthält
 - **count** ist die Gesamtzahl der Zellen im Grid, d. h. **Breite * Höhe**



Um diese Funktion aufzurufen, beachten Sie die [RISCV](#)-Aufrufkonvention. Sie können dann einen Aufruf an das Label **rust_send_grid** verwenden, um auf die Funktion zuzugreifen.

3. Schliesslich tauschen Sie die Grids **A** und **B** aus, um die Berechnung der nächsten Generation vorzubereiten, indem Sie die Hilfsfunktion `fn rust_swap_grids()`; aufrufen.



Um diese Funktion aufzurufen, beachten Sie die [RISCV](#)-Aufrufkonvention. Sie können dann einen Aufruf an das Label **rust_swap_grids** verwenden, um auf die Funktion zuzugreifen.



1. Zeichnen Sie ein Flussdiagramm, das die Schritte zur Berechnung der nächsten Generation des Grids darstellt.
2. Berechnen Sie die nächste Generation des Grids und senden Sie sie über [UART](#).

3.3 Kompilieren und flashen

Um die GoL-Software zu veröffentlichen, verwenden Sie die angegebene **justfile**, um den [RISCV](#)-Kern zu kompilieren und zu flashen:

```
# One by one
just gol-flash # Compile and flash the GoL software to the RISCV core
just gol-viewer # Launch the GoL Viewer to visualize the grid

# Both together
just run # Compile and flash the GoL software to the RISCV core,
         # then launch the GoL Viewer
```



3.4 Tests

Aufgrund der begrenzten Debugging-Funktionen - siehe [Abschnitt 1](#), nur vollständige Software-tests können automatisiert werden - siehe [Abschnitt 3.4.1](#). Der Rest des Codes kann mit den während der Labore gesehenen Tools implementiert werden oder indem der [Central Processing Unit \(CPU\)](#) geflasht und die Ergebnisse im Viewer überprüft werden.

3.4.1 HITL

Die abschliessenden Tests werden in einer [HITL](#) -Umgebung durchgeführt, in der der [RISCV](#) -Kern die GoL-Software ausführt, während eine dedizierte PC-Anwendung verwendet wird, um Ergebnisse über [UART](#) festzulegen und abzurufen und die Korrektheit der Software zu bewerten.

Um diese Tests auszuführen, verwenden Sie die angegebene **justfile**, um die [HITL](#) -Umgebung zu starten:

```
just test <port> # Compile and flash the #gls("riscv") core then launch the HITL
application to test the GoL software
# Port on Windows should be COMx, on Unix /dev/ttyUSBx or /dev/ttyACMx.
# You can also set the DEF_PORT variable in the justfile to avoid specifying the port
each time.
```

Diese Tests überprüfen:

1. Wenn frisch geflasht, sollte das Grid bereits ein Standardmuster enthalten, das einen Blinker in der Mitte des Grids enthält
2. Dann sendet es bekannte Grids und überprüft einige Generationen:
 - Block (statisches Muster)
 - Beehive (statisches Muster)
 - Blinker (Perioden-2-Oszillator)
 - Pulsar (Perioden-3-Oszillator)
 - Glider (bewegliches Muster)

Nach den Tests werden Protokolldateien neben der **justfile** generiert, um detaillierte Berichte über die Tests und deren Ergebnisse sowie eine Zusammenfassung der Tests bereitzustellen:

```
Total tests: 6
Passed: 6
Failed: 0
grid_init: PASS
block: PASS
beehive: PASS
blinker: PASS
pulsar: PASS
glider: PASS
```

```
HIL summary started: port=COM3,
baud=115200 (summary log:
<path>\<to>\test_summary.log)
grid_init log:
<path>\<to>\test_grid_init.log
grid_init PASS
pattern=block PASS
pattern=beehive PASS
pattern=blinker PASS
pattern=pulsar PASS
pattern=glider PASS
HIL result: PASS (5 passed, 0 failed)
```



3.5 Verwendung des GoL Viewers

Der GoL Viewer ist eine einfache Anwendung, um das Grid und seine Entwicklung zu definieren und zu visualisieren. Es ist in **rust** geschrieben und verbindet sich über **UART** mit dem **CPU**. Es ist für Windows, Linux und MacOS verfügbar.

Der Viewer wird automatisch aus den Rezepten **just run** oder **just gol-viewer** gestartet. Es kann auch manuell gestartet werden, indem die ausführbare Datei **gol-viewer** im Ordner **GoLViewer** des Projekts ausgeführt wird:

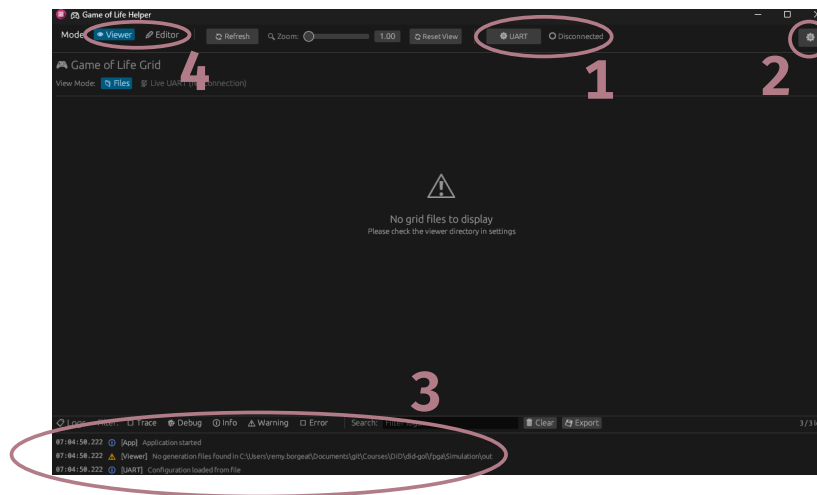


Abbildung 12 - GoL Viewer

1. **UART** -Verbindungsstatus und -konfiguration
2. Viewer- und Grid-Einstellungen
3. Protokollfenster
4. Ansicht / Bearbeitung Umschalter

3.5.1 Einrichtung

Schliessen Sie zunächst das ESP-Board an und stellen Sie dann den richtigen **UART** -Port ein. Ohne spezielle Konfiguration sollte der größte Port in der Liste der richtige sein. Stellen Sie die Baudrate auf **115'200** und die Konfiguration wie unten gezeigt ein, und klicken Sie dann auf **Connect**:

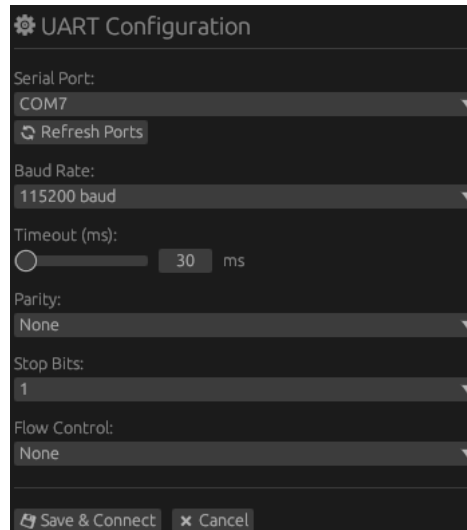


Abbildung 13 - UART Konfiguration

Stellen Sie dann sicher, dass der Viewer auf die richtige Grid-Größe eingestellt ist, die standardmäßig **32x32** beträgt.

Sie können auch Eingabe- und Ausgabeverzeichnisse festlegen, um Ergebnisse aus Dateien zu lesen und in Dateien zu schreiben (optional).

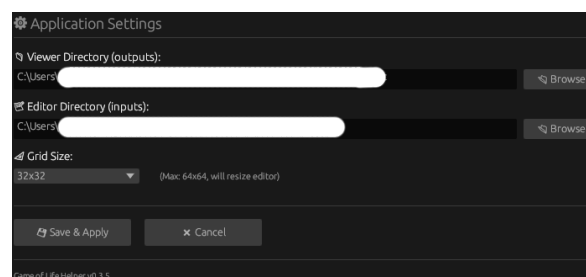


Abbildung 14 - Grid-Konfiguration

3.5.2 Anzeige des Grids

Wechseln Sie in den **Viewer**-Modus und wählen Sie die Option **Live UART**, um Grids zu sehen, die vom ESP-Board generiert werden:

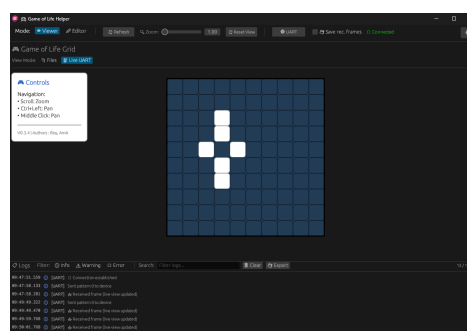


Abbildung 15 - Live-Ansicht

3.5.3 Bearbeitung des Grids

Wechseln Sie in den **Editor**, um ein benutzerdefiniertes Grid zu erstellen.

Klicken Sie einfach mit der linken Maustaste auf Zellen, um deren Zustand auf lebendig zu setzen,



klicken Sie mit der rechten Maustaste, um sie auf tot zu setzen.

Klicken Sie auf die Schaltfläche **Classic Mode**, um den **Color Mode** zu aktivieren. Wählen Sie eine Farbe aus und setzen Sie die Zellen wie gewohnt. Diese Farben werden im ersten Frame angezeigt und bei Ihrer Implementierung von **gol_step** [Abschnitt 3.2.2](#) übernommen.

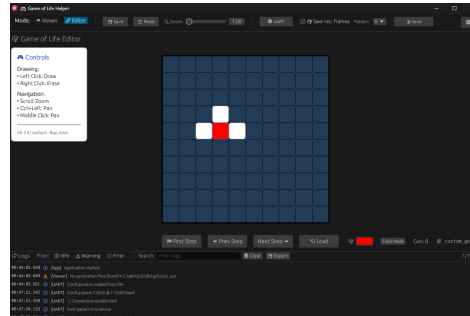


Abbildung 16 - Editor

Klicken Sie auf die Schaltfläche **Send** in der oberen rechten Ecke des Viewers, um das Grid zu senden. Gehen Sie zurück in den **Viewer**-Modus [Abschnitt 3.5.2](#), um die Entwicklung des Grids zu sehen.



4 | Bewertung

Die Schlussnote beinhaltet den Bericht, den Code sowie eine Präsentation eurerseits des Systems [Tabelle 1](#).

Evaluierte Aspekte	Punkte
Bericht	50
Einleitung	2
Spezifikation	2
Entwurf	10
Verifizierung und Validation	10
Aktivitätsdiagramm	20
Schlussfolgerung	2
Formale Aspekte des Berichtes	4
Funktionalität der Schaltung	20
Qualität der Lösung	20
Präsentation	20
Total	110

Tabelle 1 - Bewertungsraster



Das Bewertungsraster gibt bereits Hinweise über die Struktur des Berichtes. Für einen guten Bericht konsultieren Sie das Dokument **Wie verfasst man einen Projektbericht?** [\[1\]](#).



Anhang

1 | Debugging (Optional)

1.1 Einrichtung

Obwohl es nicht die nahtlose Erfahrung ist, ist es möglich, die GoL-Software zu debuggen, indem man den Assemblercode durchgeht und die Registeraktualisierungen beobachtet. Dazu ist bereits eine VSCode-Konfiguration im Projekt eingerichtet, die die [Probe-rs Debugger](#)-Erweiterung verwendet.



Es ist möglich, eine Kombination anderer Editoren und Viewer zu verwenden, entweder mit **probe-rs** oder **OpenOCD**, um den [RISCV](#) -Kern zu debuggen. Dies wird jedoch in dieser Dokumentation nicht behandelt.

1.1.1 Erweiterungen

Installieren Sie die [Probe-rs Debugger](#)-Erweiterung in VSCode, um das Debuggen des [RISCV](#) -Kerns zu aktivieren.

1.1.2 Treiber

Die Treiber müssen korrekt eingerichtet sein, um über die USB-JTAG/Seriell-Debug-Einheit auf das ESP32 zuzugreifen. Auch wenn das ESP32 korrekt über [UART](#) mit dem Host-PC kommuniziert, verwendet der Debugger den **JTAG**-Port, um mit dem Kern zu interagieren.

Für Unix-Systeme kann es sofort funktionieren. Bei Bedarf siehe [Probe-rs Einrichtung](#) für weitere Informationen.

Für Windows-Benutzer ist es erforderlich, den ESP32-**USB JTAG/serial debug unit**-Treiber über [Zadig](#) zu ändern. Der **WinUSB v.6.1**-Treiber ist anstelle des Standardtreibers erforderlich.

Öffnen Sie **Zadig**, wählen Sie **Optionen > Alle Geräte auflisten**, dann wählen Sie **USB JTAG/serial debug unit (Interface 2)**. Klicken Sie auf das Dropdown-Menü neben der Schaltfläche **> Install device** und klicken Sie dann auf **Replace Driver**:

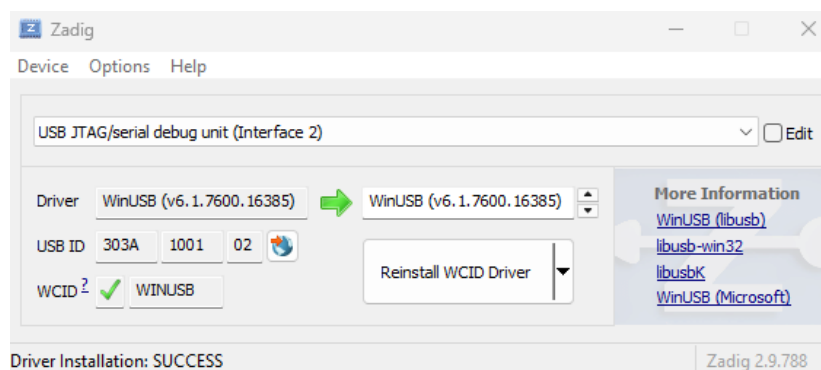


Abbildung 17 - Zadig-Treiberinstallation



1.1.3 Debug

Sobald alles eingerichtet ist, schliessen Sie das Board an und:

- Öffnen Sie den Viewer mit **just gol-viewer**
- Verbinden Sie den Viewer mit dem richtigen **UART** -Port - dies ermöglicht das Drucken von Nachrichten in der Viewer-Konsole vom **CPU** aus

Siehe Abbildung [Abbildung 19](#):

1. Öffnen Sie den VSCode-Debugger im linken Bereich:
2. Wählen Sie die Konfiguration **Debug GoL** und klicken Sie auf die grüne Wiedergabetaste, um den Debugger zu erstellen und zu verbinden
3. Setzen Sie Haltepunkte im Assemblercode in der Datei **GoLRS/src/bin/main.rs** vor der Zeile **gol_step**, indem Sie auf der linken Seite der Zeilennummer klicken (gleiche Vorgehensweise für **gol_init**, falls Sie die Initialisierung des Rasters debuggen möchten)
 - Löschen Sie die Protokolle in der Viewer-Konsole
4. Klicken Sie auf die blaue -Taste, um den Code auszuführen, bis der Haltepunkt erreicht ist
 - Überprüfen Sie die Nachrichten im Viewer und notieren Sie die angegebenen Adressen - diese entsprechen Ihrem **ASM** -Codebereich im erstellten Binärdatei

```
11:04:33.745 [CPU] Initialising grid
11:04:33.745 [CPU] * cur_ptr = 0x3fc82e74, nxt_ptr = 0x3fc83e74
11:04:33.745 [CPU] * ASM region = 0x4201014c..0x42010494
11:04:33.745 [CPU] * gol_init = 0x42010168
11:04:33.745 [CPU] * gol_step = 0x42010390
11:04:33.745 [CPU] * Initialising grid with blinker oscillator in the centre
11:04:33.745 [CPU] Running GOL main loop
```

Abbildung 18 - Viewer log

5. Öffnen Sie das Bedienfeld mit **Ctrl+Shift+P** und führen Sie **Open Disassembly View** aus

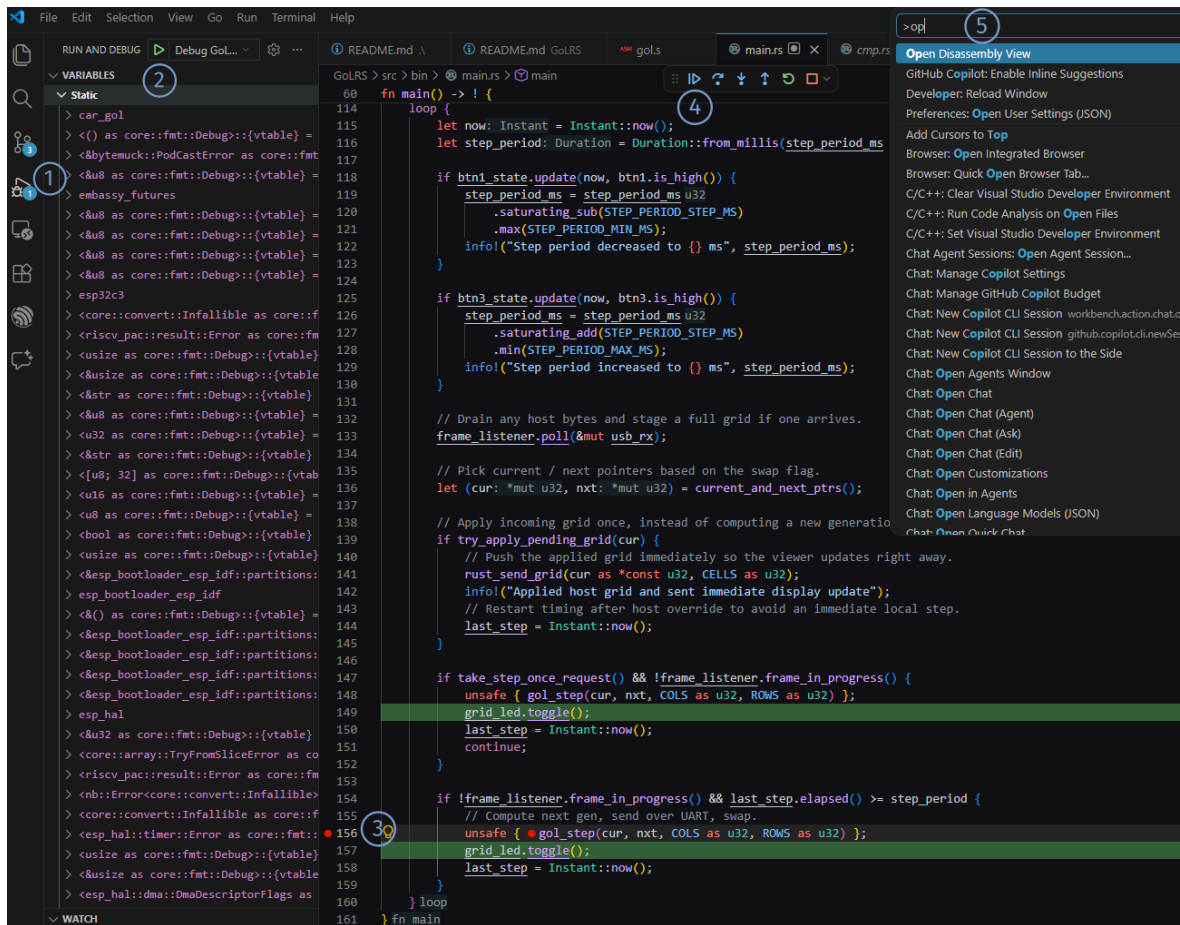


Abbildung 19 - VSCode Debugger

Dies öffnet ein neues Panel mit etwas Assemblercode. Es entspricht noch nicht der Funktion, die Sie geschrieben haben. Sie müssen noch einige Schritte durchlaufen, bis Sie die nächste **jal**-Anweisung erreichen. Klicken Sie entweder auf den Pfeil **Step Into** oder auf **F11**:

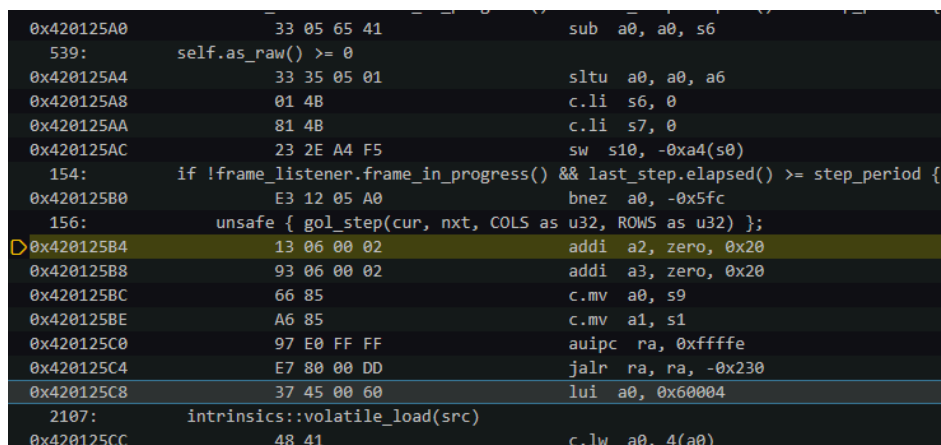


Abbildung 20 - Disassembly view

Wenn Sie die **jal**-Anweisung passieren, sollte Ihre Funktion erreicht werden. Das linke Panel zeigt die Registerwerte an:



```

VARIABLES
  Static
    <&str as core::fmt::Debug>::{vtable} 0x4201035c
    <riscv_pac::result::Error as core::fm 0x42010360
    <usize as core::fmt::Debug>::{vtable} 0x42010364
    <&usize as core::fmt::Debug>::{vtable} 0x42010368
    <&str as core::fmt::Debug>::{vtable} 0x42010370
    <riscv_rt 0x42010374
    0x42010378
    0x4201037C
    0x42010380
    0x42010384
    0x42010388
    67 80 00 00 ret
    0x42010390
    0x42010394
    0x42010398
    0x4201039C
    0x420103A0
    0x420103A4
    0x420103A8
    0x420103AC
    0x420103B0
    0x420103B4
    0x420103B8
    93 84 05 00 mv s1, a1
    0x420103BC
    0x420103C0
    0x420103C4
    0x420103C8
    0x420103CC
    0x420103D0
    0x420103D4
    0x420103D8
    0x420103DC
    0x420103E0
    0x420103E4
    0x420103E8
    0x420103EC
    0x420103F0
    0x420103F4
    0x420103F8
    0x420103FC
    0x42010400
    0x42010404
    0x42010408
    0x4201040C
    0x42010410
    0x42010414
    0x42010418
    0x4201041C
    0x42010420
    0x42010424
    0x42010428
    0x4201042C

Registers
  x0/zero = 0x00000000
  x1/LR = 0x420125c8
  x2/SP = 0x3fccc2d0
  x3/gp = 0x3fc8127c
  x4/tp = 0x00000000
  x5/t0 = 0x00ffffff
  x6/t1 = 0x00000000
  x7/t2 = 0x00000003
  x8/FP/s0 = 0x3fc82e74
  x9/s1 = 0x3fc83e74
  x10/a0/r0 = 0x3fc82e74
  x11/a1/r1 = 0x3fc83e74
  x12/a2 = 0x00000020
  x13/a3 = 0x00000020
  x14/a4 = 0x00000000
  x15/a5 = 0x00000000
  x16/a6 = 0x000f4240
  x17/a7 = 0x3fc80d0c
  x18/s2 = 0x00000000
  x19/s3 = 0x60023000
  x20/s4 = 0x60043000
  x21/s5 = 0x3fc81c74
  x22/s6 = 0x00000000
  x23/s7 = 0x00000000
  x24/s8 = 0x00000000
  x25/s9 = 0x3fc82e74
  x26/s10 = 0x000003e8
  x27/s11 = 0x00001000
  x28/t3 = 0x00000001
  x29/t4 = 0x3fc84e70
  x30/t5 = 0x00000000
  x31/t6 = 0x00000000
  pc/PC = 0x420103bc

WATCH
  0x42010420
  0x42010424
  0x42010428
  0x4201042C

```

Abbildung 21 - Step-by-step execution



Stellen Sie sicher, dass die Adressen dem Bereich entsprechen, der vom GoL-Viewer-Protokoll angegeben wird. Labels existieren nicht in der Disassembly-Ansicht.

Sie können dann durch den Code schrittweise gehen und die Registeraktualisierungen beobachten.



Glossar

ASM – Assembly [9](#), [18](#)

CPU – Central Processing Unit [12](#), [13](#), [18](#)

GoL – Game of Life [3](#), [6](#), [7](#)

HITL – Hardware In The Loop [2](#), [12](#)

ISA – Instruction Set Architecture [5](#)

LED – Light Emitting Diode [3](#)

RGB – Red Green Blue [9](#)

RISCV – 5-stages Reduced Instruction Set Computer architecture, open-sourced [4](#), [11](#), [12](#), [17](#)

UART – Universal Asynchronous Receiver Transmitter [11](#), [12](#), [13](#), [14](#), [17](#), [18](#)



Literatur

- [1] Christophe Bianchi, François Corthay, und Silvan Zahno, „Comment Rédiger Un Rapport de Projet?“. 2021.